

Flutter JSON Serializable Guide

Required Packages

```
dependencies:  
  json_annotation: ^4.9.0  
  
dev_dependencies:  
  build_runner: ^2.4.8  
  json_serializable: ^6.8.0
```

Example Model

```
// lib/models/user_model.dart  
import 'package:json_annotation/json_annotation.dart';  
import '../entities/user.dart';  
  
part 'user_model.g.dart';  
  
@JsonSerializable()  
class UserModel {  
  final int id;  
  final String name;  
  final String email;  
  
  UserModel({  
    required this.id,  
    required this.name,  
    required this.email,  
  });  
  
  factory UserModel.fromJson(Map<String, dynamic> json) =>  
    _$UserModelFromJson(json);  
  
  Map<String, dynamic> toJson() => _$UserModelToJson(this);  
}
```

Generate JSON Code

Run build_runner:
flutter pub run build_runner build --delete-conflicting-outputs
➡ This generates user_model.g.dart.
➡ Never edit .g.dart files manually.

Handle Nested Models

```
{  
  "id": 1,  
  "name": "John",  
  "profile": {  
    "age": 25,  
    "city": "Berlin"  
  }  
}  
  
@JsonSerializable()  
class ProfileModel {  
  final int age;  
  final String city;  
  
  ProfileModel({required this.age, required this.city});
```

```

    factory ProfileModel.fromJson(Map<String, dynamic> json) =>
        _$ProfileModelFromJson(json);

    Map<String, dynamic> toJson() => _$ProfileModelToJson(this);
}

@JsonSerializable(explicitToJson: true)
class UserModel {
    final int id;
    final String name;
    final ProfileModel profile;

    UserModel({required this.id, required this.name, required this.profile});

    factory UserModel.fromJson(Map<String, dynamic> json) =>
        _$UserModelFromJson(json);

    Map<String, dynamic> toJson() => _$UserModelToJson(this);
}

```

■ Update Process (When API Changes)

- 1 Add/remove/rename fields in your Model class.
- 2 Re-run build: flutter pub run build_runner build --delete-conflicting-outputs
- 3 Update the toEntity() method if necessary.
- 4 Done ■

■ Best Practices

- 1 Entities should never depend on JSON.
- 2 Always use explicitToJson: true when nested models exist.
- 3 Use JsonKey to handle different API field names:
- 4 @JsonKey(name: 'user_name') final String name;
- 5 Use @JsonSerializable(includeIfNull: false) to exclude null fields from toJson().
- 6 Always re-run codegen after any change in models.

■ Benefits Over Manual Parsing

- 1 Saves time & avoids repetitive boilerplate.
- 2 Safer → less chance of runtime errors.
- 3 Scales easily with large/complex APIs.
- 4 Keeps Entities (business logic) clean & decoupled from API.